

Understanding architectural knowledge sharing in AGSD teams: An empirical study

Gilberto Borrego

Facultad de Ciencias
Universidad Autónoma de Baja California
Ensenada, México
gilberto.borrego@uabc.edu.mx

Ramón Palacio

Departamento de Computación y Diseño
Instituto Tecnológico de Sonora
Navojoa, México
ramon.palacio@itson.edu.mx

Alberto L. Morán

Facultad de Ciencias
Universidad Autónoma de Baja California
Ensenada, México
alberto.moran@uabc.edu.mx

Oscar M. Rodríguez

División de Investigación y Posgrado
Instituto Tecnológico de Hermosillo
Hermosillo, México
omrodriguez@ith.mx

Abstract— Nowadays, the use of agile methodologies (AM) in Global Software Development (GSD) – known as AGSD – is increasingly common. However, AM and GSD are not completely compatible. On the one hand, in AM people interactions (face-to-face) are preferred over document-based communications to share knowledge. On the other hand, in GSD knowledge sharing is conducted through documents to minimize the effect of the inherent four distances (physical, temporal, language and cultural). This means that tacit knowledge is preferred in AM and explicit knowledge is preferred in GSD. These differences between AM and GSD affect many aspects of software development, for instance: Architectural Knowledge Management. According to the literature, in AGSD it is preferred to convey Architectural Knowledge (AK) by frequent interactions across sites through unstructured and textual electronic media (UTEM) (chats, emails, forums, etc.), that is, AK is articulated in these media. UTEM leave a textual record of the transmitted information, thus leaving an unstructured log of the shared AK of the project. In this paper we present an empirical study to understand AK articulation in UTEM in AGSD teams. Our results consist of an ontology that represents the involved aspects in AK articulation in UTEM in AGSD teams. Additionally, we identified eleven categories of interactions across sites through UTEM, where requirements and coding themes are prominent. Finally, we found that AK in UTEM is perceived as important, regardless the interaction frequency. These results lead us to think that a tool to structure and exploit AK in UTEM is needed in AGSD, in order to bridge the gap between AM and GSD.

Keywords— agile global software development; virtual teams; architectural knowledge; unstructured and textual electronic media.

I. INTRODUCTION

In the last years, many companies where global software development is applied have adopted agile methodologies (AM) [1], i.e., they use an agile global software development (AGSD) paradigm. In AGSD, communication between local

and remote members is particularly important. According to the agile manifesto [2], in AM people interactions are preferred over following processes, and software that actually works is preferred over comprehensive documentation. This means that the documentation produced is only that which the team considers sufficient to understand the software being developed, as in AM it is emphasized that knowledge be transferred through interactions rather than through documents. Therefore, tacit knowledge predominates over explicit knowledge in AM.

Also, geographic distribution in AGSD increases the need for a more explicit communication [3]. In addition, in GSD most of the knowledge is explicit (with formal documents and strict process tracking) to reduce the impact of the four distances of this approach (i.e. temporal, physical, cultural and linguistic) [4]; while in AM there are few documents, which are generally ad-hoc or informal. All this shows an important gap between AM and GSD (see Table I).

It is known that knowledge transfer in AGSD is limited [5], therefore one of the significant challenges in AGSD is knowledge management (KM) [6], [7]. An important kind of knowledge handled in AGSD relates to software architecture (SA). SA can be defined as “the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them” [8]. SA knowledge or architectural knowledge (AK), not only refers to structures, software components, properties and relations, but also to the SA decisions that led to their current organization [9]. It could be said that AK

TABLE I. COMPARISON BETWEEN GSD AND AM (BASED ON [4], [31])

Global Software Development	Agile Methodologies
Geographically distributed teams	Collocated teams
Many documents (formal and based on standards)	Few documentation (informal or ad-hoc)
Focused on process tracking	Focused on people and their interactions
Different culture and time zone	Same culture and time zone

management (AKM) [10] is also a challenge in AGSD, since AM reduce the time to write proper documentation, which negatively affects the coding phases [11].

In AGSD, communication between local and remote team members is performed through phone calls, instant messaging, email, and video calls, etc. [12], i.e., they share knowledge through electronic media. Some of these communication means leave a textual record of the information transmitted through them, for example, in the log files of e-mail and instant messenger. In these media (which we will call unstructured textual electronic media – UTEM), and given the agile nature of the processes, there will be AK recorded that will not be probably recorded anywhere else. It can be said that tacit AK is articulated in UTEM (according to SECI Model [13]). Thus, these records could be valuable to complement and contextualize the informal and ad-hoc documents used in AM. This could reduce the problems about the lack of explicit and clear documentation in AM, including: waste of time answering the same questions and finding solutions to problems that had been submitted previously, loss of knowledge when a project member retires, defects in software evolution and maintenance, lack of visibility for project monitoring and technical solutions and poorly understood requirements [14], [15].

This paper presents the results of an empirical study conducted in four Mexican AGSD companies. We observed UTEM interactions between local and remote team members, and face-to-face interactions between local members, where AK issues were addressed. The main goal of the study is to understand and characterize AK articulation through UTEM in an AGSD environment. The rest of the paper is organized as follows: in Section II, we present related work, Section III presents a description of the study conducted, while in Section IV we present its results. In Section V we present a discussion of the results and finally in Section VI we present our conclusions and directions of future work.

II. RELATED WORK

Since KM was identified as a challenge in AGSD, there have been efforts to understand how agile teams communicate in distributed settings. In order to identify the related work of this study, we conducted a literature review considering two main topics: KM in AGSD and AKM in AGSD.

Firstly, AGSD communications have been studied from a distributed cognition perspective [16], and it was found that local members heavily rely on knowledge codification, aiming at spreading it out to remote members through Web tools. This result was confirmed by Razzak and Smite [17], and additionally they mapped common KM practices in AGSD to standard KM strategies, called knowledge schools [18]. In addition, Dorairaj et al. [6] identified several practices in terms of knowledge generation, codification, transfer and application, in order to contribute to the successful implementation of AGSD projects. In another study [19], the importance of showing videoconferences on public displays to reduce the effect of GSD's distances was highlighted. Further, Boden et al. [20] showed that public displays in AGSD environments help to encourage ad-hoc coordination and decision-making processes.

On top of that, given that communication across sites is part of the base of KM strategies in AGSD, Juarez et al. [21] explored the use of social networks as a source of knowledge. By monitoring Facebook interactions on an agile and distributed team, they found that team members share technical knowledge, i.e., AK is shared through this social network. However, it was not reported what kind of AK was shared. The high occurrence of comments about AK makes us believe that constant communication through Facebook was used to mitigate the lack of technical documents.

Regarding the works related to AKM in AGSD, taking into account that AGSD and GSD share common characteristics, Clerc et al. [22] validated AKM practices commonly used in GSD in an AGSD environment. They obtained that half of the validated practices were commonly used in AGSD, highlighting the following practices: (1) Frequent interaction between sites using electronic media (video/audio calls, chats, emails, etc.); (2) knowing who is who, i.e., knowing who the right people for each subject are. Surprisingly, the least common practices were more related to architectural design issues.

Additionally, other studies have characterized problems related to SA and processes in AGSD [23], [24]. In short, these problems are characterized as follows: process-centric coordination, communication and integration cost, and interaction problems. Also, in the same studies, an architecture-centric paradigm was proposed to add efficiency and productivity to software cycles in AGSD, with support from Web 2.0 technologies.

Recapping, the use of Web tools (UTEM - chats, emails, social media, etc.) is part of KM strategies in AGSD environments, since these tools are suitable to share codified knowledge across sites. Even, these tools have been proposed as a support for an architecture-centric approach in AGSD. However, it has been found that AKM practices that promote personalization get much more attention than those promoting centralization in AGSD [22]. This means that AGSD workers prefer to establish contact with a team member to get AK (usually through UTEM), instead of writing SA documents and sharing them through a common repository.

Although the literature shows that AK is shared through UTEM in AGSD, to the best of our knowledge, until the present day, there is no complete understanding of AK sharing across distributed sites in AGSD environments, particularly regarding UTEM.

III. STUDY DESCRIPTION

In this section we describe an empirical study involving four Mexican AGSD companies. The study consisted of an on-site observation on one of the four companies and an exploration of observations on all the four companies.

A. Objective

To understand AK articulation in UTEM between local and remote members in AGSD teams, in order to have elements to technologically support AKM in distributed and agile teams.

B. Research questions

- *RQ1* - What elements are involved in AK articulation in AGSD teams?
- *RQ2* - Which are the most frequent AK topics in UTEM interactions in AGSD teams according to team members' perception?
- *RQ3* - How are the perceived incidence and the perceived importance of AK topics in UTEM interactions among AGSD team members related?
- *RQ4* - How are the AK topics in UTEM and face-to-face interactions, and AK documents related among them?

C. Participants

Study participants were 20 software engineers working in 4 different Mexican AGSD companies (see Table II). Participants stay in contact with remote counterparts through UTEM.

The participating companies have projects with others in the US, Spain, Ecuador, Argentina, Colombia, Peru, Guatemala and Honduras. All the companies apply Scrum as methodological base to manage projects. However, regarding requirements management, company B and company D only use user stories; company A uses a mix of user stories and features; and company C receives requirement documents from its clients, but they have to be refined on the way. Regarding software design and construction, the four companies share the following practices: spike solutions and agile modeling. Company B and company C also apply pair programming and participative design agile practices.

Regarding work distribution, participants of company B and company D, are part of development teams where remote developers are involved. Thus, these companies share responsibility in all the development cycle between local and remote members. Also, they only have face-to-face design meetings in case of a complex project. In the case of company A, the participants refine requirements with their remote teammates, and they have the product and development control. Also, participants of company A conduct system testing and develop deployment documents. Their remote teammates are responsible for user testing and software deployment. Finally, participants of company C receive requirements documents and develop, test and deploy the software product.

TABLE II. PARTICIPANTS CHARACTERISTICS PER COMPANY (AVG = AVERAGE, MED = MEDIAN)

Company	Number of participants	Participating projects	Age ^a		Software engineering experience ^a		AGSD experience ^a	
			Avg	Med	Avg	Med	Avg	Med
A	11	6	23.7	23	1.5	1.5	0.9	0.4
B	4	3	36.3	35.5	13.5	13	5.3	5.5
C	3	3	26.3	27	3	3	1.6	1
D	2	2	28.5	28.5	7	7	4.5	4.5

^a. Amounts expressed in years.

D. Data gathering

Data gathering was divided in two phases: observation and exploration. The observation phase was conducted only in company A in an on-site manner, given that it was the only company that granted us permission to monitor and analyze their UTEM interactions. We conducted observation during 3 weeks, 5 days a week and 8 hour per day. Six projects and eleven participants were involved, where interactions with team members from Spain and Latin America were performed.

The main intention of the observation phase was to register and analyze the UTEM interactions across sites. For this purpose, we asked company A participants for the most common UTEM that they use to interact with local and remote members. The following media was included: IM (Microsoft Lync), email (Microsoft Exchange) and discussion forums (part of the Target Process tool). The interactions through these media were captured with an *ex-professo* custom developed software tool.

We decided to observe face-to-face interactions among collocated members, and to analyze the documentation artifacts related to SA aspects, in order to find a relation between them and UTEM interactions. Face-to-face interactions were registered in a log, including: beginning and ending time, supporting media (computer, notebook, board, etc.), participant name and a brief description of the interaction topic. It is worth saying that we did not interact with the participants during the observation, we only registered the observation in the log.

In order to have a reference model to identify when an architectural topic was discussed, either in interactions or documents, we used the 4+1 views architectural model [25]. This model establishes five architectural views, which are described next:

- *Scenarios view*: considers requirements, use cases, user stories, etc.;
- *Logical view*: abstractions and key mechanisms, responsibilities separation and distribution;
- *Implementation view*: classes, components and code, subsystems;
- *Process view*: components and classes interactions at the business process level;
- *Deployment view*: hardware, configuration, and deployment issues.

During the exploration phase, we analyzed the findings of the observation phase along with the participants of the four companies. We conducted four focus group sessions (one per each company). The findings were summarized in (i) UTEM interactions, (ii) a list of artifacts used to record architectural issues, and (iii) the main intentions of face-to-face interactions. The audio of the focus groups sessions was recorded.

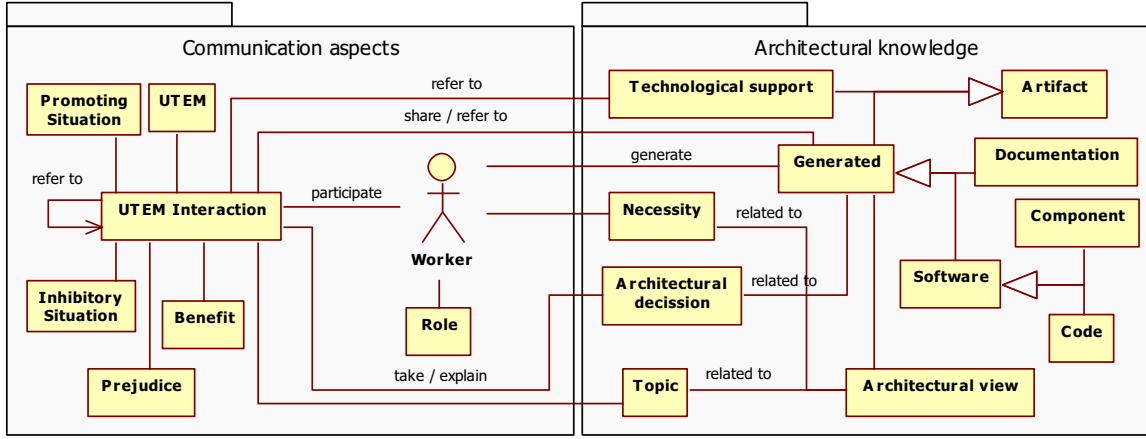


Figure 1. Ontology that represents the involved aspects in AK articulation through UTEM in AGSD teams (all relations without label are of type “have”).

E. Data analysis

To analyze UTEM interactions, participants of company A were asked for a list of keywords used in each project to refer to architectural issues, both formally and informally. This list was then used to filter UTEM interactions in order to keep only AK related interactions. Once filtered, interactions were read one by one to identify the main SA topics.

In order to analyze face-to-face interactions each entry in the log was examined to obtain the main contact intentions among the local team members.

Finally, the focus groups audios were transcribed and Grounded Theory was applied in order to understand AK articulation through UTEM in AGSD teams.

IV. RESULTS

The results of this study are presented in two subsections: Overall understanding of AK articulation in UTEM, and UTEM and face-to-face interactions.

A. Overall understanding of AK articulation in UTEM

During the observation phase we registered 45 face-to-face interactions of different SA aspects. Also, 216 UTEM interactions were recorded and analyzed, where 145 were from IM conversations, 51 from email messages and 20 from discussion forums comments. It is worth noting that, MS Lync defines an IM conversation as a group of sent messages while an IM window stays opened.

Based on these registered interactions in company A, and on the focus group sessions with all the participating companies, we developed an ontology (see Figure 1) that represents our understanding of AK articulation through UTEM in AGSD teams, with all the elements that we could identify (this fulfills *RQ1*). The resulting ontology has two parts: communications aspects and AK aspects. We describe them in the following paragraphs.

Regarding communication aspects, the main element is a UTEM interaction related to AK sharing among AGSD workers. The involved AGSD workers play a specific role in

the team (developer, tester, analyst, product owner, client, etc.). We noted that AGSD workers frequently refer to a past interaction (IM conversation, email message, forum entry, etc.), in order to argue or give context to the current discussion.

We found that UTEM interactions related to AK are promoted or triggered by different situations. Some of these promoting situations are the following: ask someone for specific information (through UTEM) to avoid wasting time looking for AK in documents; a team receives a new member and a remote team gives him AK context; poor or nonexistent AK documentation (AK is asked through UTEM); doubts on legacy systems and complex code components (experts are consulted through UTEM).

We also found situations that inhibit AK interactions. Some of these situations are the following: referencing repositories (instead of extending an UTEM interaction to share AK), strict adherence to well-defined methodologies (avoiding misunderstandings and frequent questions), following documentation best practices (e.g. explicit documents and code conventions), avoid asking a question to the same person several times (avoiding feeling ashamed while asking).

Additionally, we identified benefits and prejudices of establishing AK related interactions through UTEM. Some of the identified benefits were: requirements refinement, knowledge transfer among team members, and decreased number of software errors (bugs), among the most cited by participants. Furthermore, the most cited prejudices were: wasted time and a sense of knowledge unreliability (since knowledge comes from informal media such as IM and forum entries).

Concerning AK aspects, we observed that artifacts are involved during UTEM interactions, either generated or as technological support artifacts. Also, AGSD workers refer to technological support artifacts (e.g. IDE, bug trackers, tasks managers, repositories, etc.) during UTEM interactions. Regarding generated artifacts, these are shared or referred in

TABLE III. UTEM INTERACTIONS CATEGORIES RELATED TO ARCHITECTURAL VIEWS OF 4+1 ARCHITECTURAL MODEL (S = SCENARIOS, L = LOGIC, I = IMPLEMENTATION, P = PROCESSES, D = DEPLOYMENT)

Categories	Architectural views				
	S	L	I	P	D
1. Code segments sharing			x		
2. Testing/deployments environment settings					x
3. Information request about the system's internal flow			x	x	
4. Communicate information about classes (without code sharing)			x		
5. Referencing business rules or internal operation that was explained by a third party	x		x	x	
6. Technical explanation about the solution of errors			x		
7. Business rules, features or user stories clarification	x				
8. New project proposals	x				
9. User level doubts about the functioning of a finished product	x				
10. Database topics clarification		x	x		x
11. Information request about structuring data for a test	x				

UTEM interactions by AGSD workers. This kind of artifacts include: documentation (formal or ad-hoc diagrams, specifications, schedules, etc.) and software, either components (executable, libraries, frameworks, etc.) or source code snippets or files. All these generated artifacts are related to a particular architectural view, according to their nature.

It is worth highlighting that in the participating companies, most of the architectural documents were related to requirements specification. Technical documentation is almost absent. These documents are usually realized on specific request only.

In addition, we observed that when a generated artifact is shared or referred, an explanation is associated to it. Architectural decisions are occasionally present in that explanation. For example: when a software artifact is shared, the sender usually explains how it works, and why it has to work in that manner. Also, architectural decisions could be taken through UTEM, depending of the work distribution of the AGSD team. For instance, when all the team members are developers (local and remote) it is more probable that this kind of decisions were taken using UTEM.

Additional AK aspects that we identified were knowledge needs in a particular architectural view. For example, participants manifested the following AK needs: diagrams, access to architectural design, access to e-mail and IM history (to look for AK), and know whom to ask specific architectural issues. Finally, we identified different architectural topics in UTEM interactions (e.g. coding,

requirements, databases, testing, etc.) that are also related to different architectural views.

B. UTEM and face-to-face interactions categorization

Once a preliminary understanding of AK articulation in UTEM was achieved, and aiming at fulfilling *RQ2*, we analyzed the captured UTEM interactions in order to categorize the AK topics that are discussed in these media (the same topics identified as part of AK in the ontology presented above). We obtained eleven UTEM interaction categories (see Table III), which were related to different architectural views of the 4+1 views model. For example, Category 3 ("Information request about the system's internal flow") was related to the implementation and processes views because their interactions were referred to coding and components interaction. It is worth clarifying the relationship of categories 8 ("New project proposals") and 10 ("Databases topics clarification") with architectural views. Category 8 was related to the scenarios view, since interactions were about overall functionality for new projects. Category 10 was related to the implementation and deployment views, because its interactions were focused on databases querying, configuration and installation.

One of the highlights of the interactions was that participants commented that most of them repeat information. Whether by oversight or because the message was not well understood since the beginning, they turned to discuss some aspects through UTEM. In this regard, one of

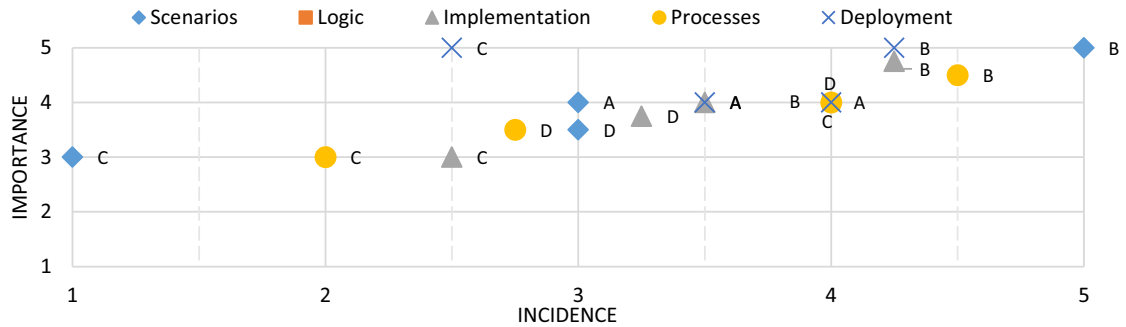


Figure 2. Likert valuation in terms of importance and incidence of the UTEM interactions, grouped by architectural views and company (A, B, C, D). Obtained from the participants evaluation of the 11 categories of the UTEM interactions (Note: logic view symbol is hidden behind the cumulus located on the point (4,4)).

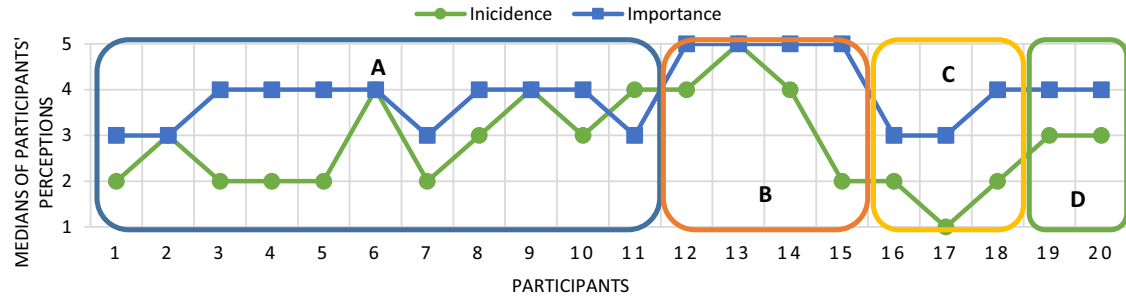


Figure 3. Median of perceived importance and perceived incidence in Likert Scale obtained from each participant (company A = blue box, company B = orange box, company C = yellow box, company D = green box).

the participants in the focus groups mentioned: “... you may cause a bad impression to the other person if you are continuously asking questions ...”.

Another important aspect is that the scenarios and implementation views comprise many of the categories (see Table III). In addition, only category 10 is related to the logical view. This may mean that few issues of conceptual elements are discussed in UTEM interactions (in this case database schemas). Instead, programming requirements topics were the most predominant aspects.

In order to identify how frequent and how important the identified categories are, we conducted an evaluation where all participants assessed the categories in those terms according to their personal perceptions. A Likert scale structured as follows was used: 1 – very rare/unimportant, 2 - rare/unimportant, 3 - neutral, 4 - common/important 5 - very common/important. We grouped the obtained responses by architectural view and by company, and we obtained the medians for each group.

To compare them, we created a scattered graph, where the X axis corresponds to incidence (frequency) values and the Y axis corresponds to importance values (see Figure 2). We observed that most of the architectural views were perceived at least as “Important” (more than 3 in the Likert Scale). In contrast, in terms of incidence, there is more variability in the perceptions. However, approximately half of the incidence perceptions were perceived at least as “Common” (more than 3 in the Likert Scale).

Furthermore, we visually note a difference between the incidence and importance of UTEM interactions. There is more dispersion in incidence perceptions than in importance perceptions. In order to confirm this difference in an analytical manner, all the participants were took as a single sample of AGSD workers. We considered that it is valid to group the

participants in that manner, since all of them share AK through UTEM regardless their work style; and precisely their style is reflected in the dispersion of incidence and importance. Subsequently, the median of perceived importance and perceived incidence was obtained from each participant. Thus, two sets of data were obtained (graphically seen in Figure 3), where $n_1 = n_2 = 20$, i.e., the total number of participants in this study.

Given the sample size and nature of data, we decided to apply a Wilcoxon signed-rank test (a non-parametric test). We defined the null hypothesis (H_0) as follows: There is no difference between the incidence and importance perceptions of the UTEM interactions related to AK. The alternative hypothesis (H_1) was: The perceived importance is significantly greater than the incidence perception of the UTEM interactions related to AK. After applying the statistical test, with a significance level of 0.01 (see details in Table IV), the null hypothesis ($p\text{-value} = 0.00062 \leq 0.01$, $W \leq W_{p \leq 0.01}$) was rejected. Therefore, we can establish that the importance perception of UTEM interactions is greater than the incidence perception for the participants of this study (this fulfills $RQ3$).

On the other hand, also face-to-face interactions were observed in order to have the complete scenario of the AK related interactions in an AGSD environment. As we mentioned earlier, we registered 45 face-to-face interactions in company A. We could highlight that 39% of these interactions were related to clarifications about current functionality, and 29% to current technical operation (see Table V). This means that programming and requirements topics are prominent in face-to-face interactions.

Further, if we group the interactions by architectural view (see Figure 4) considering the 4+1 views architectural model,

TABLE IV. WILCOXON SIGNED-RANK TEST DETAILS (EFFECTIVE SAMPLE SIZE $N = 16$)

W statistic values		Z statistic values	
W-value	5.5	Z-value	-3.2318
Mean Difference	-0.19	Mean (W)	68
Sum of positive ranks	5.5	Standard Deviation (W)	19.34
Sum of negative ranks	130.5	p-value	0.00062
$W_{p \leq 0.01} (n = 16)^a$	23		

^a Effective sample size is $n=16$ because there are four pairs of perceptions of incidence and importance that are equals (see Figure 3).

TABLE V. FACE-TO-FACE INTERACTIONS RELATED TO ARCHITECTURAL VIEWS

Face-to-Face AK topics	Incidence percentage	Architectural view
Current functionality clarifications	39%	Scenarios
Current technical operation clarifications	29%	Implementation
New functionality clarifications	16%	Scenarios
Database programming clarifications	11%	Implementation
Application/environment settings	5%	Deployment

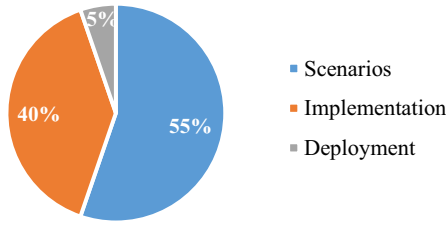


Figure 4. Face-to-face interactions incidence grouped by architectural view.

we observe that 55% of them were related to the scenarios view, and the rest were related to the implementation (40%) and deployment (5%) views. From this perspective, we may see a coincidence between UTEM and face-to-face interactions, as these three architectural views (scenarios, implementation and deployment) were frequent in UTEM interactions.

Additionally, given that observation was performed only in company A, during the focus group sessions we asked the participants from the other companies for their most frequent architectural topics in face-to-face interactions. The reported topics were: functionality clarification, technical operation of any application (both in a general way and in detail), and technical support on development issues. These topics were consistent with those observed in company A.

V. DISCUSSION

The discussion of results is organized three-fold: firstly we discuss UTEM interaction categories and companies' work style; then we discuss AK documentation, UTEM interactions and face-to-face interactions; and finally, we present a discussion about problems related to AK sharing.

A. UTEM interactions categories and companies' work style

Regarding UTEM interactions categories, it was initially considered that the four companies could have a similar behavior in terms of incidence and importance, as they work using the AGSD paradigm. However, when looking at a more detailed level, we noticed that each company has a particular behavior. For instance, if we compare the perceptions of company B and company C (see Figure 2), their perceptions are located in opposite sides of the incidence axis.

On the one hand, considering the case of company B, we notice that the perceptions on both incidence and importance for all categories are high (see Figure 5). This suggests that knowledge is shared more by interactions than by documents. Particularly, in this company, it stands out that most of the participants work on projects with remote counterparts, who are also developers like them; thus they are part of a virtual team dedicated to coding. This is reflected in the incidence and importance behavior of company B (see Figure 5), as the categories with the highest values in both perceptions are related to the scenarios (C5, C7, C11), implementation (C3, C10) and deployment (C10) views. In other words, clarifications are common and important on the following topics: requirements (C7), databases (C10), test case structuring (C11), technical operations (C3), and business rules referenced by a third party (C5).

On the other hand, considering the case of company C, we noticed that the perception of both incidence and importance for most categories tended to be low (see Figure 5). This behavior could mean that company C does not share much AK through UTEM interactions. In fact, participants of this company usually receive explicit and extensive requirements documents (up to 30 pages), and their doubts are clarified preferably by phone. In addition, their counterparts usually only establish requirements and they expect a finished software product. This is reflected in categories related to implementation (C1, C3, C4, C5, C6, C10) and deployment views (C2, C10), which are those with the lowest incidence values.

In general, we identified that the perception of incidence and importance of the eleven categories may reflect the companies' work style. For example: company A reported that they rarely have developers as remote counterparts, however, their counterparts are testers and application installers. Therefore, categories related to testing and application deployments (C2 and C10) are the highest in incidence and importance (just below categories related to requirements). In addition, company D reported that they have developers as counterparts, and that they use repositories as the primary means for sharing technical knowledge. This is reflected in categories related to implementation view (C1, C2, C4 and C10), because they are the highest on incidence in this company (again, just below categories related to requirements). However, categories related to implementation

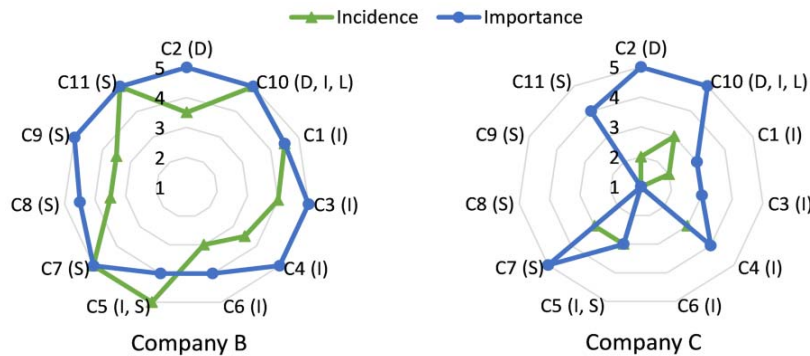


Figure 5. Company B and company C perceptions (expressed in Likert scale) of UTEM interaction categories along with the corresponding architectural view (S = Scenarios, L = Logic, I = Implementation, P = Process, D = Deployment)

view are not considered as important by company D, as its participants are confident in their ability to derive technical issues, either from source code, or from database schemas.

Furthermore, we identified that there is a dependency between working style and the way the company uses UTEM to share AK, which in turn shows the impact of this kind of media on AGSD [26]. Also, it is worth noting that even though companies presented different ways to distribute the work, most of the categories were related to the scenarios and implementation views, and were rated as the most frequent and the most important ones.

The abovementioned also shows the relevance of the topics discussed through UTEM. We obtained that the perceived importance of the identified categories was significantly higher than the interaction incidence. This means that knowledge shared through UTEM is important, regardless of the sharing frequency and working style.

B. AK documentation, UTEM interactions and face-to-face interactions

During the observation phase, we only found documents related to the scenarios view, i.e., documents related to features specification and user stories. Participants commented that requirements documents are generally the unique base to the whole development cycle. Also, they commented that they write technical documents only on request. Regarding this situation, one participant said that: *"... the only time I had to write a document is because there will be a meeting with the client..."*.

Additionally, participants commented that they rarely write technical documents because these kind of document are rarely consulted: *"... we generate UML diagrams, but we rarely see them, unless we do not understand the code..."*. Particularly, company D heavily relies on the discipline and technical skills of its developers. They mentioned that self-documenting code (i.e. assigning descriptive names to variables, methods, classes, etc.) is a common practice.

On the other hand, participants of company B said that they write technical diagrams, but only as an aid in problem understanding. These diagrams are usually ad-hoc and "disposable", i.e., they are not kept as project documentation.

This may be influenced by the fact that none of the six observed projects was in the early stages, and therefore the conceptual issues have already been addressed. However, company A reported that they work only based on the Scrum methodology, which does not include a phase to identify conceptual elements for software development [27]. It could be said that this behavior reflects the nature of the agile paradigm [2], in which design time tends to be reduced or even disappeared.

Considering the UTEM interaction categories, it is noticeable that most of the categories related to requirements (C5, C7 and C11) were perceived as the highest in incidence and importance in the four participating companies. Also, 55% of face-to-face interactions were related to the scenarios view (see Figure 3). Although requirements were the only documented architectural element, this behavior shows that these documents are unclear. In sum, this means that when an architectural view is not properly documented, it increases the

interaction among the team members, both face-to-face and through UTEM (this fulfills *RQ4*). Additionally, it reflects the application of two agile manifesto values [2]: "Individuals and interactions over processes and tools" and "Working software over comprehensive documentation". The former results in the feature of few documentation or an inexistent documentation in agile teams as also evidenced by [28], [29].

C. Problems related to AK sharing

Considering the point of architectural documents, it was found that the absence or lack of clarity in them, also encourages interactions consultation [30]. In this particular study, this was reflected in the leading causes of UTEM interactions. In principle, preferring working software and individuals' interaction is beneficial for reducing delivery times [31]. In fact, most of the participants mentioned that consulting doubts through UTEM is a quick path to AK, which was perceived as a benefit for them. However, there must exist a high availability and willingness of people, in order not to become an obstacle [32]. In other words, inter-organizational communication has a big influence on development speed [30].

In the case of AGSD teams, remote members are available only a few hours a day, regarding the time zone difference and the daily work activities. This reduces the available time to clarify issues between local and remote members (at least synchronously). For example, one of the participants reported the following regarding the availability of their counterparts: *"...sometimes I have days of delay... because the analyst is absent..."*. On the other hand, although remote members were fully available, some participants consider that they have to spend up to three hours a day on average doing requirements clarification activities, either by phone call or UTEM (part of the prejudices shown in Figure 1).

Another negative consequence of sharing knowledge through personal interactions (by means of UTEM or face-to-face) is AK vaporization [33]. This means that knowledge could be altered or lost through time, when knowledge is not properly codified. In the participating companies AK vaporization affects when new members are integrated into the teams, or when previous projects are taken up, as there is insufficient AK in documents. Then, architectural documents should be complemented by peer interactions (local or remote). Peer interactions do not guarantee the full acquisition of the needed knowledge, since it depends on the memory and knowledge of colleagues.

On this regard one of the participants commented: *"... I had to browse a lot in the code ... struggle a lot to understand how to make a small change in the application and I think I still do not know it well enough yet..."*. This is, the lack of AK caused that a simple tasks took longer than planned. This confirms the importance of knowing at least the reference architecture in AGSD teams [30].

Related to the time factor, one of the participants said: *"... I have spent too much time in bug fixing... because I had so many questions for my counterpart that I preferred to stop..."*. One of the situations that inhibit UTEM interactions is that it could be embarrassing to ask too much through UTEM, or it could be annoying to receive too many questions. This could

lead to a detriment of social relationships among team members, and may affect the knowledge flow (especially tacit) in agile teams [34].

In the above case, the participant is more concerned with knowledge of the implementation view. While the categories related to the implementation view (C1, C3, C4, C6 and C10) were not perceived as frequent as those related to the scenarios view, the importance of these interactions is high, as they can adversely affect software development.

On the other hand, the results of this study show that there is an accessibility problem regarding AK in AGSD environments. This problem is caused both by the preference to share AK through interpersonal interactions, and by the physical and temporal distances. In this paper we focus on UTEM interactions, which leave "traces" of information where AK could be extracted [26], [35]. On this regard, some participants reported efforts to recover this knowledge from email records and instant messenger conversations (as also reported in [26]). However, they do not have the mechanisms to effectively exploit these sources, i.e., currently this process consumes them so much time and effort, which should rather be spent on their substantive tasks.

In order to reduce the time and effort required for AK recovery from UTEM records, first AK should be classified. One way to classify it is based on the proposed ontological model (see Figure 1), particularly considering the AK part. On this regard, the literature [36], [37] reports that there are documentation and artifacts models that could be used to strengthen the classification of knowledge elements.

D. Threats to validity

Although some of the results obtained in this study are consistent with others reported in the literature, we should consider that we only had 20 participants from four different AGSD local companies. Also, we should consider that more than 50% of the participants work in company A (11 participants), thus the results could be skewed by their behavior. Although this evidence points out to internal validity regarding the included AGSD workers and companies, in order to gain external validity it is necessary to conduct an additional study with a larger sample size, both from workers and companies.

Also, it should be noted that the participating companies work based on the Scrum methodology, which focuses more on project management and control, rather than on software development itself. None of the companies reported a standard development methodology to support the engineering part of their software development process. That is, each company has its own set of methods, even ad-hoc, to guide their software development cycle. Thus, it would be convenient to extend this study to other AGSD companies that have a more formal developing process.

In this study we obtained a set of eleven categories of UTEM interactions related to AK, which were consistent among the four participant companies. The proposed set of categories does not intend to be comprehensive, but a starting proposal that can be reduced or extended as required.

VI. CONCLUSIONS AND FUTURE WORK

In this paper we presented the results of an empirical study conducted with four AGSD Mexican companies. We observed and analyzed different aspects related to software architecture in an AGSD environment: artifacts, unstructured text electronic media (UTEM) and face-to-face interactions. We used as a reference the 4+1 views architectural model in order to understand and characterize the architectural artifacts and the UTEM and face-to-face interactions related to AK.

On the one hand, we found that scenarios were the only architectural view that is documented. Also, we found that face-to-face interactions related to the scenarios view were the most frequent, because the scenarios documents were generally incomplete or not clear. Additionally, face-to-face interactions related to the implementation view were the second in frequency. This reflects at least two agile values in the AGSD teams: "Individuals and interactions over processes and tools" and "Working software over comprehensive documentation".

On the other hand, we identified eleven categories of UTEM interactions, where most of them were related to the scenarios and implementation views (coinciding with face-to-face interactions). We observed that work distribution between local and remote members is a factor that affects the perceived incidence and importance of these categories. Also, we obtained that AK shared through UTEM is important, regardless of the interaction frequency in AGSD teams. All these show us that there exists valuable AK in UTEM interactions, regardless of the work distribution between local and remote members in AGSD teams.

Furthermore, we developed an ontological model of the involved aspects in the articulation of AK in UTEM by AGSD teams. Based on this model and on the other findings of this study, we conclude that it exists a trend where agile practices are prominent in an AGSD environment, although the use of documents helps to minimize the effects of the four distances of GSD. In fact, it has been determined that the most important drivers for AGSD relate to the Agile Principles and the rest of drivers relate to global business perspectives [5].

For future work, we plan to formalize our understanding of AK articulation through UTEM in terms of knowledge management. We propose to explore the use of KoFi [38] as a methodology for this purpose, aiming at obtaining a set of design guidelines that ease the access to AK in UTEM through a software tool. There have been efforts to support agile and distributed development through social media mining [39], [40], but they only consider email messages as a source. Our intention is take email messages and instant messenger traces as the main sources.

Also, it could be convenient to explore AK shared through video/audio calls, since many participants said that usually they clarify their doubts by these means too.

Finally, this work can be considered as our first effort towards understanding AK articulation in AGSD teams, aiming at bridging the gap between agile development and GSD by means of architectural knowledge management.

ACKNOWLEDGMENT

The authors are grateful to the participating companies: EMCOR, Tiempo Development, Softek and Nearsoft, for the support provided to carry out the present study, and for their willingness to continue working with us in future projects. Also, we are grateful to CONACyT for supporting this study with scholarship 394125 for the first author.

REFERENCES

- [1] B. Ramesh, L. Cao, K. Mohan, and P. Xu, "Can distributed software development be agile?," *Commun. ACM*, vol. 49, no. 10, p. 41, Oct. 2006.
- [2] M. Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, "The Agile Manifesto." 2001.
- [3] P. Kruchten, "Contextualizing agile software development.," *J. Softw. Evol. Process*, vol. 25, no. 4, pp. 351–361, 2013.
- [4] A. Vizcaino, F. García, M. Piattini, and P. Garrido, "El desarrollo global del software," in *Desarrollo Global de Software*, A. Vizcaino, F. García, and M. Piattini, Eds. Ra-ma, 2014, pp. 37–58.
- [5] N. Kamaruddin, N. Arshad, and A. Mohamed, "Comparison of drivers between global software development and agile global software development: A SURVEY," in *Computer and Mathematical Sciences Graduates National Colloquium 2013*, At Faculty of Computer and Mathematical Sciences, UiTM Shah Ala, 2012.
- [6] S. Dorairaj, J. Noble, and P. Malik, "Knowledge Management in Distributed Agile Software Development," in *2012 Agile Conference*, 2012, pp. 64–73.
- [7] M. Jiménez, M. Piattini, and A. Vizcaino, "Challenges and Improvements in Distributed Software Development: A Systematic Review," *Adv. Softw. Eng.*, vol. 2009, no. 710971, p. 14, 2009.
- [8] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 2nd ed. Addison-Wesley Professional, 2003.
- [9] M. Babar, T. Dingsoyr, P. Lago, and H. van Vliet, *Software architecture knowledge management: theory and practice*. Springer Berlin Heidelberg, 2009.
- [10] H. Van Vliet, "Software architecture knowledge management," in *Software Engineering, 2008. ASWEC 2008. 19th ...*, 2008, pp. 24–31.
- [11] R. Noordeloos and C. Manteli, "From RUP to Scrum in Global Software Development: A Case Study," 2012.
- [12] N. Ale Ebrahim, S. Ahmed, and Z. Taha, "Virtual Teams: a Literature Review," *Australian J. Basic Appl. Sci.*, vol. 3, no. 3, pp. 2653–2669, 2009.
- [13] I. Nonaka and H. Takeuchi, *The knowledge-creating company: How japanese companies create the dynamics of innovation*. New York: Oxford University Press, 1995.
- [14] H. Holz, G. Melnik, and M. Schaa, "Knowledge management for distributed agile processes: Models, techniques, and infrastructure," in *Enabling Technologies: Infrastructure for Collaborative Enterprises, 2003. WET ICE 2003.*, 2003, pp. 291 – 294.
- [15] N. Uikey, U. Suman, and A. Ramani, "A Documented Approach in Agile Software Development," *Int. J. Softw. ...*, vol. 2, no. 2, pp. 13–22, 2011.
- [16] H. Sharp, R. Giuffrida, and G. Melnik, "Information Flow within a Dispersed Agile Team: A Distributed Cognition Perspective," in *Agile Processes in Software Engineering and Extreme Programming SE - 5*, vol. 111, C. Wohlin, Ed. Springer Berlin Heidelberg, 2012, pp. 62–76.
- [17] M. A. Razzak and D. Smite, "Knowledge Management in Globally Distributed Agile Projects – Lesson Learned," in *Global Software Engineering (ICGSE), 2015 IEEE 10th International Conference on*, 2015, pp. 81–89.
- [18] M. Earl, "Knowledge Management Strategies: Toward a Taxonomy," *J. Manag. Inf. Syst.*, vol. 18, no. 1, pp. 215–233, May 2001.
- [19] J. Garbajosa, A. Yagüe, and E. Gonzalez, "Communication in Agile Global Software Development: An Exploratory Study," in *On the Move to Meaningful Internet Systems: OTM 2014 Workshops*, Springer Berlin Heidelberg, 2014, pp. 408–417.
- [20] A. Boden, F. Rosswog, G. Stevens, and V. Wulf, "Articulation spaces: bridging the gap between formal and informal coordination," in *Proceedings of the 17th ACM ...*, 2014, pp. 1120–1130.
- [21] R. Juárez, R. Pimienta, and V. Ocegueda, "Using Social Networks for Integrating a Tacit Knowledge Repository to Support," in *Integrated Uncertainty in Knowledge Modelling and Decision Making*, Beijing, China, 2013, pp. 167–179.
- [22] V. Clerc, P. Lago, and H. Van Vliet, "Architectural Knowledge Management Practices in Agile Global Software Development," in *2011 IEEE Sixth International Conference on Global Software Engineering Workshop*, 2011, pp. 1–8.
- [23] D. Šmite, P. J. Ågerfalk, and N. B. Moe, "Architecture-Centric Development in Globally Distributed Projects," *Agil. Across Time Sp. Implement. Agil. Methods Glob. Softw. Proj.*, pp. 1–341, 2010.
- [24] D. Šmite, P. J. Ågerfalk, and N. B. Moe, "Coordination between global agile teams: from process to architecture," in *Agility Across Time and Space: Implementing Agile Methods in Global Software Projects*, 2010, pp. 1–341.
- [25] P. Kruchten, "The 4+ 1 view model of architecture," *Software, IEEE*, vol. November 1, p. 9, 1995.
- [26] R. Harrison and V. Veerappa, "Social Media Collaboration in Software Projects," in *Software Project Management in a Changing World SE - 16*, G. Ruhe and C. Wohlin, Eds. Springer Berlin Heidelberg, 2014, pp. 401–424.
- [27] K. Schwaber and J. Sutherland, "The scrum guide," *Scrum. org*, Oct., vol. 2, p. 17, 2011.
- [28] C. J. Stettina and W. Heijstek, "Necessary and neglected? An empirical study of internal documentation in agile software development teams," in *Proceedings of the 29th ACM International Conference on Design of Communication (SIGDOC 2011)*, 2011, pp. 159–166.
- [29] D. Rost, M. Naab, C. Lima, and C. von Flach García Chavez, "Architecture Documentation for Developers: A Survey," in *Software Architecture*, K. Drira, Ed. Montpellier, France: Springer Berlin Heidelberg, 2013, pp. 72–88.
- [30] A. Martini, L. Pareto, and J. Bosch, "Communication factors for speed and reuse in large-scale agile software development," *Proc. 17th Int. Softw. Prod. Line Conf. - SPLC '13*, p. 42, 2013.
- [31] A. Smith, G., Sidky, *Becoming Agile: ...in an imperfect world*. Manning, 2009.
- [32] C. Young and H. Terashima, "How did we adapt Agile processes to our distributed development?," in *Proceedings - Agile 2008 Conference*, 2008, pp. 304–309.
- [33] J. Bosch, "Software Architecture: The Next Step.," in *EWSA, 2004*, vol. 3047, pp. 194–199.
- [34] S. Ryan and R. V. O'Connor, "Acquiring and sharing tacit knowledge in software development teams: An empirical study," *Inf. Softw. Technol.*, vol. 55, no. 9, pp. 1614–1624, Sep. 2013.
- [35] A. R. Yanzer Cabral, M. B. Ribeiro, and R. P. Noll, "Knowledge Management in Agile Software Projects: A Systematic Review," *J. Inf. & Knowl. Manag.*, vol. 13, no. 01, p. 1450010, 2014.
- [36] M. Kuhrmann, D. M. Fernández, and M. Gröber, "Towards artifact models as process interfaces in distributed software projects," in *Proceedings - IEEE 8th International Conference on Global Software Engineering, ICGSE 2013*, 2013, pp. 11–20.
- [37] K. A. De Graaf, P. Liang, A. Tang, W. R. Van Hage, and H. Van Vliet, "Computers in Industry An exploratory study on ontology engineering for software architecture documentation," vol. 65, pp. 1053–1064, 2014.
- [38] O. M. Rodríguez-Elias, A. Vizcaino, A. I. Martínez-García, J. Favela, and M. Piattini, "Knowledge Flow Identification," in *Encyclopedia of Information Science and Technology*, Second Ed., IGI Global, 2009, pp. 2337–2342.
- [39] A. Moraes, E. Silva, C. da Trindade, Y. Barbosa, and S. Meira, "Recommending experts using communication history," in *2nd International Workshop on Recommendation Systems for Software Engineering - RSSE '10*, 2010, pp. 41–45.
- [40] S. M. Sohan, M. M. Richter, and F. Maurer, "Auto-tagging emails with user stories using project context," *Lect. Notes Bus. Inf. Process.*, vol. 48 LNBIP, pp. 103–116, 2010.